

Assignment 2 — Classificazione di Immagini attraverso BoW

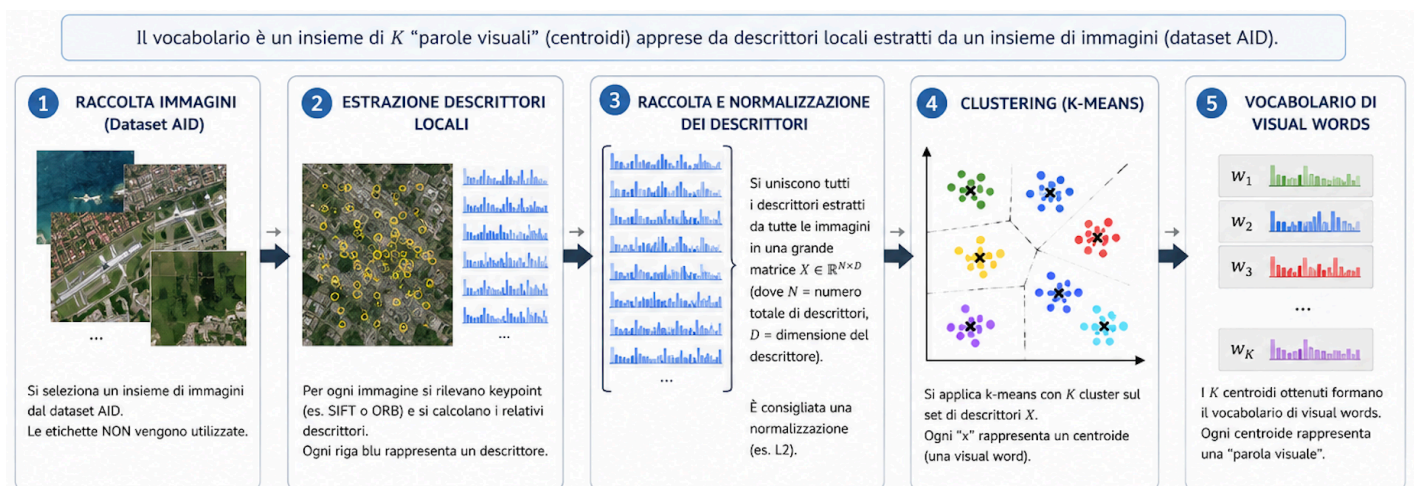
Sviluppare un sistema completo per la **classificazione di immagini aeree** acquisite da camere RGB.

Data una immagine in input, il sistema dovrà classificare l'immagine come appartenente ad una delle seguenti **21 classi**: *agricultural, airplane, baseball_diamond, beach, buildings, chaparral, dense_residential, forest, freeway, golf_course, harbor, intersection, medium_residential, mobile_homepark, overpass, parking_lot, river, runway, sparse_residential, storage_tanks, tennis_court*.



Il metodo dovrà utilizzare un approccio *Bag-of-Visual-Words* in cui un vocabolario di K “parole visuali” è addestrato a partire dai descrittori locali (es. *SIFT* o *ORB*). Quindi, ogni immagine è rappresentata attraverso un istogramma di tali *visual words*. Un opportuno classificatore shallow (per es. *SVM* o *Random Forest*) è quindi addestrato per classificare ogni immagine in una delle 21 classi.

1. Calcolo del Vocabolario di Visual Words



Utilizzare *opencv* per l'estrazione di **descrittori locali** (SIFT o ORB, a vostra scelta) dalle immagini.

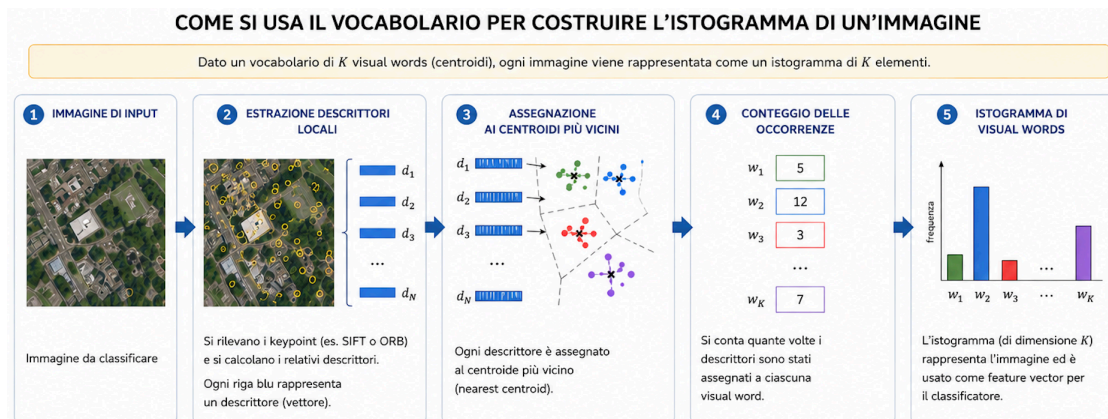
Per apprendere il vocabolario di visual words, useremo un dataset diverso da quello usato per l'addestramento del classificatore. In particolare, si usino le immagini del dataset AID (Aerial Image Dataset) che può essere scaricato dal link alla pagina <https://www.kaggle.com/datasets/jiayuanchengala/aid-scene-classification-datasets>. **Le etichette del dataset AID non devono essere utilizzate: le immagini servono esclusivamente per estrarre descrittori locali.**

Siete liberi di scegliere se limitare o meno il numero di immagini o di descrittori estratti in base alle risorse di calcolo/memoria a vostra disposizione (se estraete troppi descrittori locali e non riuscite a gestirli, potete sempre sotto-campionare i descrittori. Potreste decidere di limitare il numero di descrittori estratti a 100-200 per immagine o comunque a 100k per l'intero dataset). Applicare ai descrittori, se lo si reputa utile, una normalizzazione appropriata (L1 o L2).

Una volta che un buon numero di descrittori è disponibile, apprendere un **vocabolario di parole visuali** tramite clustering. Si usi *k-means* con k prefissato. I k centroidi rappresenteranno approssimazioni delle visual words e formeranno il vocabolario. È utile salvare i descrittori o il vocabolario sfruttando la libreria *pickle* (per evitare di dover ripetere i calcoli ogni volta).

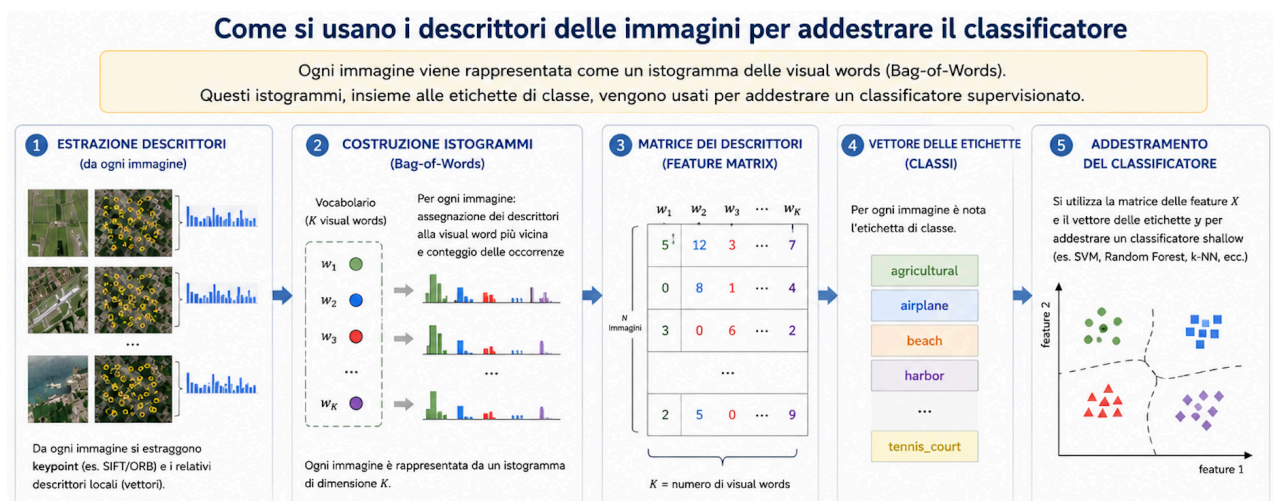
Attenzione: il vocabolario va costruito una sola volta all'inizio.

2. Rappresentazione delle immagini attraverso istogrammi di Visual Words



Fissato il vocabolario, costituito da K parole visuali, possiamo rappresentare ogni immagine attraverso un istogramma di k elementi. Per ogni immagine, estraiamo i descrittori locali, assegniamo ciascun descrittore alla parola visuale del vocabolario più vicina (*nearest centroid*), costruiamo un **istogramma delle visual words**. È consigliato normalizzare gli istogrammi (L1 o L2).

3. Dataset e Classificatore



Utilizzeremo *UC Merced Land Use Dataset* disponibile al link <https://www.kaggle.com/datasets/abdulhasibuddin/uc-merced-land-use-dataset>. Si utilizzi un **3-fold cross-validation protocol** avendo cura di lasciare il numero di immagini per ciascuna classe bilanciato. Utilizzare **almeno due** classificatori shallow (a vostra scelta) tra i seguenti: *SVM lineare*, *SVM con kernel RBF*, *Random Forest*, *k-NN*, *Logistic Regression*. Potete tunare, se volete, gli iperparametri dei classificatori per ogni fold ricavando un validation set dal training set di ogni iterazione della cross-validation (*a dire il vero, suggerisco di usare gli iper-parametri di default per semplicità e di mantenerli costanti in ogni fold*).

4. Valutazione del metodo

Utilizzare **metriche** appropriate per la **classificazione multi-classe**: *Accuracy*, *Precision*, *Recall*, *F1-score (macro-average)*, *Confusion Matrix*. Le metriche possono essere calcolate tramite *scikit-learn (classification_report, confusion_matrix)*.

Gli esperimenti in cross-validation devono permettere di:

- studiare il comportamento del sistema al variare della dimensione del vocabolario K in $\{50, 100, 500\}$ e la selezione del valore più conveniente di K (**obbligatorio**)
- selezionare il miglior classificatore tra i 2 (almeno) scelti (**obbligatorio**)
- selezionare le migliori features locali tra SIFT e ORB (**opzionale**)

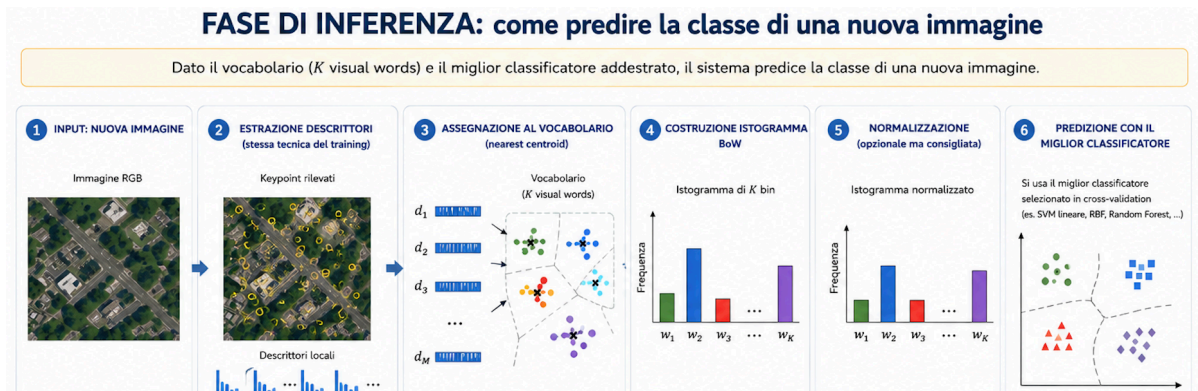
5. Inferenza

Prevedere uno script/una funzione che permetta all'utente di utilizzare il miglior modello su una nuova immagine. Dato il path di una immagine RGB in input, il sistema dovrà:

1. caricare l'immagine;
2. estrarre i descrittori locali usando la stessa tecnica scelta in fase di training;
3. assegnare i descrittori alle visual words del vocabolario appreso;
4. costruire l'istogramma BoW dell'immagine;

5. applicare la stessa eventuale normalizzazione usata in training;
6. usare il miglior classificatore selezionato in cross-validation per predire la classe dell'immagine.

È quindi richiesto di salvare e riutilizzare tutti gli elementi necessari all'inferenza, ad esempio: vocabolario di visual words; eventuali parametri di normalizzazione; miglior classificatore addestrato.



Presentazione dei risultati e Consegna

Oltre alla consegna del codice, è richiesta una breve presentazione in formato slide (PDF). La presentazione deve contenere al massimo 10 slide (esclusa una slide aggiuntiva finale) e deve descrivere in modo chiaro e sintetico:

- l'approccio utilizzato (pipeline del metodo)
- le scelte progettuali principali (feature, vocabolario, classificatori)
- eventuali problemi riscontrati durante lo sviluppo e come sono stati affrontati
- i risultati ottenuti in cross-validation e il modello selezionato
- una matrice di confusione relativa al miglior modello
- possibili miglioramenti del sistema

Slide aggiuntiva (obbligatoria)

È richiesta una slide aggiuntiva (non conteggiata nelle 10 precedenti) contenente: 1) Riferimenti 2) Elenco di eventuali risorse utilizzate (articoli, documentazione, tutorial, codice online, ecc.) 3) Uso di strumenti di Intelligenza Artificiale (sono stati utilizzati strumenti di AI (es. ChatGPT, Copilot, ecc.)? in che modo sono stati utilizzati (es. debugging, scrittura codice, comprensione teorica, ecc.))